Data Structures : Structures that store and organise data

Stack :
E.g. : stack of Plates

Works on Last In, First Out → LIFO concept

How do we define Stack ?
Syntax : stack<data_type> stack_name;
stack<int> st;

Insert from top
Removal from top

// we get the value of top element
Syntax : stack_name.top()
St = 10
20
30
st.top() = 10

// remove top element from stack

Syntax : stack_name.pop()

! -> not

Bool ok = true
!ok = false

!st.empty() = !0 = 1

```
if(!st.empty()){
    st.pop();
    Cout << st.top();
}
```

Queue : queue for a bus, queue in front of ticket counter
Works on FIFO concept -> First In, First Out

How do we define queue?
queue<data_type> queue_name;

Adding elements in a queue
queue_name.push(x) …… x-> value

Insertion from back
Removal from front

// to get the value of first element (front element)
queue_name.front()

OOPS (Object - Oriented Programming)

Class : Blueprint / design

Customer Class's Properties :

1. Name
2. Email
3. Id
4. Phone
5. Address

—> First Customer ()

Customer C1 -> Actual item / person
C1’s name = “Divya”
C1’s id = 1
C1’s email = “divya@gmail.com”
C1’s phone = “4595904355”
C1’s Address = “C-34/23 Azadpur”

Customer C2
….

Customer Cn

How do we define class ?
Syntax :
class Class_name{
public / private/ Protected :
	// Properties
	String name;
	Int age;

	// method / function

```
    // to get the name of Student
    String Student_name(){
            Return Student.name;
     }
    Int Student_age(){
            return Student.age;
     }
}
```

Book ->>> associate with Student_name();

```
Class car {   // Parent Class
    String color;
    String engine_type;
    Int seater;
}
Class Mercedes : public Car{
     // additional features

}

Class Mahindra{
```

}

E.g.
Animal -> sound
Dog -> (behave1)
Cat -> (behave2)
Camel ->
Bird->
What is polymorphism?
Same forms -> different behaviours

```
Class Animal(){      // Parent class
    Void sound(){
          Cout << “Animal makes a sound”;
    }
}

// Child class : Dog
Class Dog : public Animal{
      Void sound() override{
            Cout << “Dog barks”;
      }
}
```

```
// Constructor
Construct an object
// name, age
Student(String s, int a){
      name = s;
      age = a;
}

Student s1(“Sanya”, 19);
Student s2(“Dakshita”); // error
```

Homework :

Task - 1: Write a program to perform operations on stack

Task - 2: Write a program to perform operations on queue

Task - 3: Write a program to create a class, constructor, make few objects of that class.